Multilingual Fine-Tuning of a Small Language Model

Riccardo Corte

April 2026

Abstract

This work presents a controlled multilingual fine-tuning experiment on a small instruction-tuned language model. The model is evaluated before and after LoRA-based supervised fine-tuning in order to measure whether training changes its multilingual answering behaviour. The analysis compares exact match, token overlap, semantic similarity, response length, and Bayesian estimates of improvement. Particular attention is given to probe languages, used to assess whether the effect of fine-tuning transfers beyond the main training distribution.

1. Introduction

Large language models often display uneven behaviour across languages. Even when a model is generally capable, its answers may become less precise, less concise, or less aligned with the expected format in multilingual settings.

This project studies whether supervised fine-tuning can shift a small instruction-tuned model toward improved multilingual question-answering behaviour. The experiment is designed as a controlled comparison between a baseline model and its LoRA fine-tuned version.

The work follows a complete experimental pipeline: dataset preparation, baseline inference, supervised fine-tuning, post-training inference, and comparison between the baseline and fine-tuned models.

2. Project Presentation and Justification

The project focuses on the fine-tuning of a small instruction-tuned language model for multilingual question-answering. The chosen setup is intentionally lightweight and reproducible, using parameter-efficient fine-tuning rather than full model retraining.

The main motivation is to evaluate whether a limited fine-tuning procedure can modify specific model behaviours. In this context, the desired behaviours are:

- producing answers closer to the expected target answer;
- reducing unnecessary verbosity;
- improving consistency across multilingual inputs;
- improved performances on unseen languages.

This makes the project relevant both from a practical and methodological perspective. Practically, it reflects a common real-world scenario in which a pretrained model must be adapted to a narrower multilingual task. Methodologically, it allows the effect of fine-tuning to be analysed through a direct baseline versus fine-tuned comparison.

This study is ultimately a complete immersion inside the fine-tuning world, allowing to focus both on the training methods and on the results analysis.

2.1. Research Focus: Behaviour on Unseen Languages

A central focus of the project is the behaviour of the fine-tuned model on languages that were not the main target of training. This is important because multilingual fine-tuning can have different effects:

- it may improve the model only on the languages seen during training;
- it may generalize to related languages;
- it may degrade performance on languages outside the training distribution;
- it may mostly affect answer style, length, or concision rather than factual correctness.

For this reason, the evaluation includes probe languages such as Spanish and Portuguese. These languages are useful for testing whether the fine-tuning procedure produces cross-lingual generalization rather than narrow memorization of the training distribution.

The comparison on unseen languages is not interpreted as a definitive benchmark of multilingual ability. Instead, it is used as a diagnostic tool to understand whether the learned adaptation transfers to linguistically related but held-out settings.

3. Dataset Choice

The dataset used in this project consists of multilingual instruction-style question-answering examples. The data was filtered and organized by language, then split into training, validation, and test subsets.

The dataset choice is motivated by three main requirements:

- **Multilingual coverage**: the dataset must contain examples across different languages.
- **Instruction-style format**: the examples should resemble the input-output structure expected by instruction-tuned models.
- **Simple evaluation**: the target answers should allow automatic comparison through exact-match and length-based metrics.

The data was converted into a supervised fine-tuning format, where each example contains a user prompt and an expected assistant response. This format matches the conversational template used by the selected instruction model.

The final dataset organization follows the structure:

- training split for supervised fine-tuning ($\sim$80%);
- validation split for monitoring training ($\sim$10%);
- test split for final evaluation ($\sim$10%);
- probe splits for unseen-language evaluation.

4. Baseline Inference

Before fine-tuning, the base model is evaluated on the same test and probe datasets used later for comparison. This step establishes the baseline behaviour of the pretrained instruction model.

For each example, the input prompt is formatted using the model chat template. The

model then generates an answer under deterministic decoding settings, so that comparisons are reproducible.

The baseline inference stage produces a set of predictions of the form:

$$\hat{y}_{\text{base}} = f_\theta(x) \tag{1}$$

where x is the input prompt, f_θ is the pretrained model, and $\hat{y}_{\text{base}}$ is the generated baseline answer.

The baseline outputs are saved and later merged with the fine-tuned outputs using the example index and language code.

5. Training Procedure

The model is fine-tuned using supervised fine-tuning with LoRA adapters. LoRA, or Low-Rank Adaptation, allows the model to learn task-specific updates without modifying all pretrained parameters.

Instead of updating the full parameter set θ, LoRA introduces a small number of trainable low-rank matrices. This makes training more memory-efficient and suitable for limited GPU resources.

The fine-tuned model can be represented schematically as:

$$f_{\theta+\Delta\theta_{\text{LoRA}}}, \tag{2}$$

where $\Delta\theta_{\text{LoRA}}$ denotes the adapter-based parameter update learned during training.

The main advantages of this approach are:

- reduced memory usage;
- faster experimentation;
- feasibility on small-scale compute resources.

Training is monitored using validation loss. The best checkpoint is selected according to the lowest validation loss and then used for final inference.

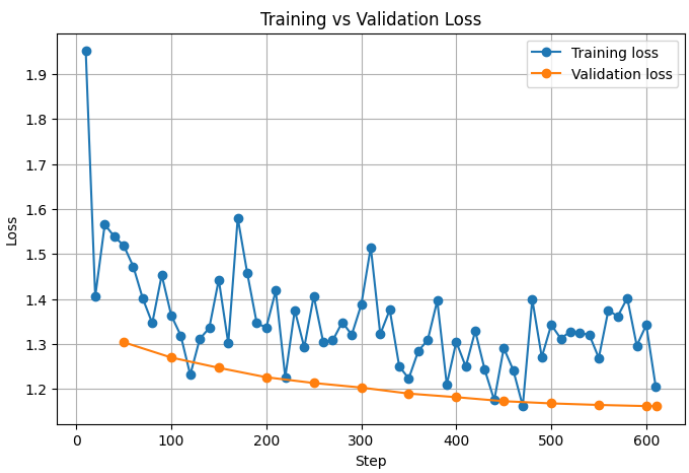

Figure 1. Training and evaluation loss during supervised fine-tuning. The training curve tracks the optimization dynamics, while the evaluation loss monitors generalization on the validation split. The comparison is used to check whether the model continues to improve without clear signs of overfitting.

5.1. Training Specifics

Model We fine-tune the instruction-tuned transformer *Qwen2.5-3B-Instruct*, a 3B parameter model optimized for chat-style generation. A medium-scale model was chosen because it enables controlled experimentation on a single GPU while still exhibiting non-trivial multilingual capabilities.

Hardware and Environment Training is performed on a single NVIDIA T4 GPU (16 GB VRAM) within a cloud-based VM (CloudVeneto). Most design choices (batch size, optimizer configuration, and parameter-efficient fine-tuning) are constrained by this hardware setting.

Training Strategy As hinted, rather than full fine-tuning, we adopt **LoRA (Low-Rank Adaptation)**. This approach freezes the base model weights and introduces trainable low-rank updates to selected linear layers. Concretely, a weight matrix $W \in \mathbb{R}^{d\times k}$ is decomposed as:

$$W \to W + \Delta W, \quad \Delta W = AB,$$

where $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{r\times k}$ with $r \ll \min(d, k)$.

This reduces the number of trainable parameters by several orders of magnitude and makes training feasible within the memory constraints of a single T4 GPU.

Why LoRA instead of full fine-tuning Full fine-tuning would require storing gradients and optimizer states for all ∼3B parameters, exceeding the available VRAM. LoRA allows us to isolate these effects by restricting updates to a low-dimensional subspace.

Optimization Parameters

Parameter	Value
Learning rate	2×10^{-4}
Optimizer	AdamW
Batch size (per device)	1
Gradient accumulation steps	8
Effective batch size	8
Number of epochs	1
Max steps	∼512
Logging steps	10
Evaluation steps	50
Save steps	50
Save total limit	2

Justification of optimization choices

- **Learning rate (2×10^{-4})** Parameter-efficient fine-tuning typically tolerates higher learning rates than full fine-tuning, since only a small subset of parameters is updated. The chosen value lies in the standard LoRA range (10^{-4} — 3×10^{-4}) and was selected to balance convergence speed and stability. Lower values resulted in slower adaptation, while higher values led to occasional instability in early steps.

- **Optimizer (AdamW)** AdamW is standard for transformer fine-tuning due to its adaptive learning rates and decoupled weight decay, which improves generalization and stabilizes training in low-batch regimes.
- **Batch size and gradient accumulation** Due to VRAM limitations, the per-device batch size is fixed to 1. To compensate, we use gradient accumulation over 8 steps, yielding an effective batch size of 8. This improves gradient estimation without increasing memory usage.
- **Number of epochs (1)** The dataset size and objective (behavioral adaptation rather than full retraining) make multiple epochs unnecessary and potentially harmful (overfitting). Empirically, most of the performance gain occurs within the first epoch in LoRA-based fine-tuning. More epochs could be used to furtherly optimize the model but weren't necessary for the goal of this study.
- **Max steps (∼600)** This corresponds to a single pass over the dataset with the chosen batch configuration. It ensures sufficient exposure to the data while keeping training time bounded.
- **Evaluation and checkpointing** Frequent evaluation (every 50 steps) allows monitoring convergence, while checkpointing with a limit of 2 avoids excessive disk usage without losing the best model.

LoRA Configuration

Parameter	Value
Rank (r)	16
Alpha	32
Dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj

Justification of LoRA design

- **Rank ($r = 16$).** The rank controls the dimensionality of the update subspace. A small rank enforces strong regularization, while a larger rank increases expressive power. The choice $r = 16$ was a compromise: sufficiently expressive to capture meaningful adaptations, yet small enough to maintain efficiency in the chosen setup.
- **Scaling factor (alpha = 32).** The scaling factor controls the magnitude of the low-rank update via:
$$\Delta W = \frac{\alpha}{r} AB.$$
Setting $\alpha = 2r$ ensures that updates are neither too small (underfitting) nor too large (destabilizing training).
- **Dropout (0.05).** A small dropout is applied to LoRA layers to reduce overfitting, particularly important in low-data or single-epoch regimes.
- **Target modules.** We apply LoRA to both attention and MLP projections:
  - `q_proj, k_proj, v_proj, o_proj`: these control attention dynamics and are critical for adapting how information is routed across tokens.

- `gate_proj, up_proj, down_proj`: these correspond to the feed-forward (MLP) blocks, which are responsible for nonlinear transformations and feature mixing.

Restricting updates to these linear layers is motivated by the observation that most of the model's expressive power lies in these projections. Empirically, adapting them is sufficient to induce significant behavioral changes without modifying the full parameter space.

Design Philosophy This setup design will affect directly *behavioral adaptation* due to its low-rank updates, so the focus will be on detecting the overall increased capacity of the model.

6. Fine-Tuned Inference

After training, the fine-tuned model is evaluated on the same datasets used for baseline inference. This produces a second set of predictions:

$$\hat{y}_{\text{ft}} = f_{\theta+\Delta\theta_{\text{LoRA}}}(x), \tag{3}$$

where $\hat{y}_{\text{ft}}$ is the answer generated by the fine-tuned model.

Using the same prompts, datasets, and decoding setup ensures that the comparison isolates the effect of fine-tuning as much as possible.

The final comparison is therefore performed example by example:

$$(x_i, y_i, \hat{y}_{\text{base},i}, \hat{y}_{\text{ft},i}), \tag{4}$$

where y_i is the target answer, $\hat{y}_{\text{base},i}$ is the baseline prediction, and $\hat{y}_{\text{ft},i}$ is the fine-tuned prediction.

7. Metric Definition

Evaluating generative language model outputs is not straightforward, since a prediction can be correct while differing from the reference answer in wording, length, or formatting. For this reason, the evaluation uses a combination of strict, approximate, semantic, and diagnostic metrics.

7.1. Exact Match

The first metric is exact-match accuracy after light normalization. Both the target answer and the generated prediction are normalized by lowercasing and stripping leading or trailing whitespace. For a prediction $\hat{y}_i$ and target answer y_i, the exact-match score is defined as

$$\text{EM}_i = \begin{cases} 1, & \text{if } \text{norm}(\hat{y}_i) = \text{norm}(y_i), \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

The dataset-level exact-match accuracy is then

$$\text{EM} = \frac{1}{N} \sum_{i=1}^{N} \text{EM}_i. \tag{6}$$

Exact match is intentionally strict. It is useful for measuring whether the model reproduces

the expected short-form answer, but it can underestimate performance when the model gives a correct answer with a slightly different formulation.

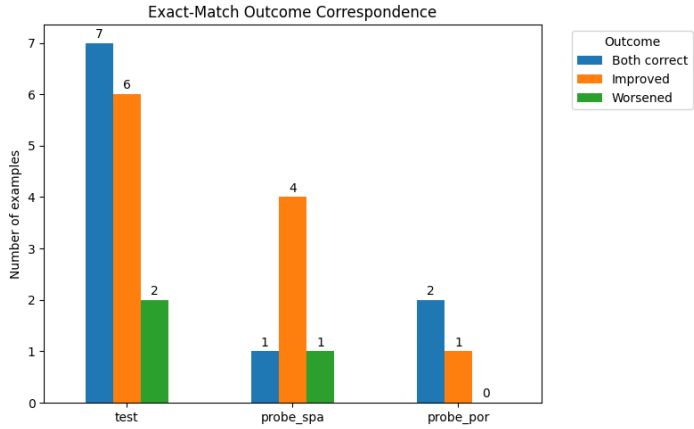

Figure 2. Exact-match outcome correspondence between the baseline and fine-tuned model. Each category describes how an example changes after fine-tuning: both models correct, both models incorrect, improved, or worsened. This view separates genuine improvements from cases where fine-tuning introduces new errors.

7.1.1. Token Overlap

To obtain a softer lexical comparison, we compute token overlap between the prediction and the target. After normalization and tokenization, the overlap is measured using the Jaccard similarity between the two token sets:

$$J(\hat{y}_i, y_i) = \frac{|T(\hat{y}_i) \cap T(y_i)|}{|T(\hat{y}_i) \cup T(y_i)|}, \tag{7}$$

where $T(\cdot)$ denotes the set of normalized tokens.

This metric captures cases where the model uses many of the same relevant words as the target answer, even if the complete answer is not an exact match.

7.1.2. Semantic Similarity

Exact match and token overlap are still limited because they operate mostly at the surface level. A prediction may be semantically close to the target while using different wording. To capture this, we compute semantic similarity using sentence embeddings.

Let $e(\hat{y}_i)$ and $e(y_i)$ be the embedding vectors of the prediction and target. Their semantic similarity is measured with cosine similarity:

$$S(\hat{y}_i, y_i) = \frac{e(\hat{y}_i) \cdot e(y_i)}{\|e(\hat{y}_i)\| \|e(y_i)\|}. \tag{8}$$

This metric provides a softer estimate of whether the prediction is close to the target in meaning.

7.1.3. Hybrid Approximate Correctness

Token overlap and semantic similarity are then combined to define a more flexible notion of approximate correctness. The goal is to identify predictions that are not exact matches but are still likely to be acceptable answers.

This hybrid criterion is more flexible than exact matching while remaining more conserva-

tive than embedding similarity alone. It is used to identify a set of approximately appropriate answers based on both lexical overlap and semantic proximity.

However, the metric is not a definitive correctness measure. It can fail in cases where two answers are lexically or semantically close but factually opposite, such as "yes" versus "no", or "increase" versus "decrease". For this reason, hybrid similarity is interpreted as a diagnostic signal rather than as a replacement for manual evaluation.

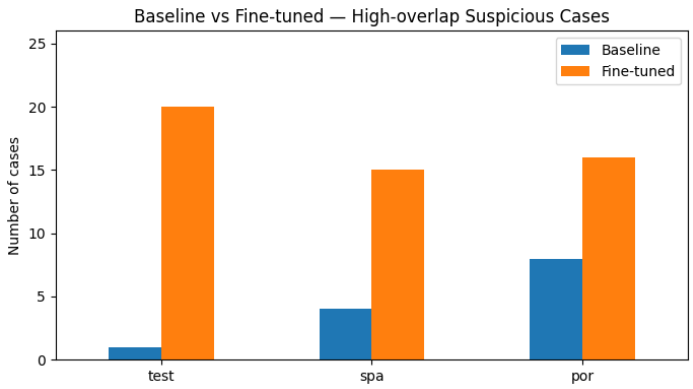

Figure 3. High-overlap suspicious cases for the baseline and fine-tuned models. These are examples with high lexical or semantic similarity to the target answer but potentially incorrect meaning. The plot is used as a diagnostic check for cases where automatic similarity metrics may overestimate correctness.

7.2. Length-Based Diagnostics

The evaluation also tracks the length of the generated answers. For each prediction, length is defined as the number of whitespace-separated words:

$$L(\hat{y}_i) = \text{number of words in } \hat{y}_i. \tag{9}$$

We then compute the mean absolute length error with respect to the target answer:

$$\text{MALE} = \frac{1}{N} \sum_{i=1}^{N} \left| L(\hat{y}_i) - L(y_i) \right|. \tag{10}$$

The fine-tuned model produces shorter answers than the baseline across all datasets. Table 1 reports the mean absolute length error between predictions and target answers.

Dataset	Baseline	Fine-tuned
Test	55.08	51.11
Spanish probe	38.94	31.06
Portuguese probe	28.10	23.41

Table 1. Mean absolute length error between generated predictions and target answers.

The decrease in length error suggests that fine-tuning makes the model more concise and closer to the expected answer format. This should not be interpreted as correctness. Rather, it measures verbosity and format alignment.

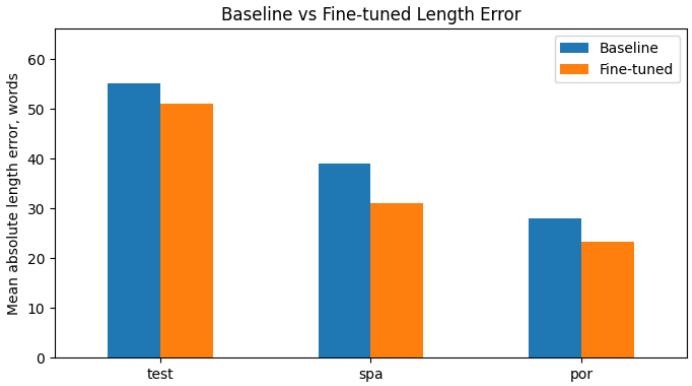

Figure 4. Mean absolute length error for baseline and fine-tuned predictions. Length error is computed as the absolute difference between the number of words in the prediction and in the reference answer. Lower values indicate better alignment with the expected answer format, although this metric should not be interpreted as correctness.

8. Bayesian Success Analysis

To quantify uncertainty in the comparison between the baseline and fine-tuned model, we perform a Bayesian analysis of the observed successes and failures.

For each metric, we model the true success probability of the baseline model and the fine-tuned model as Bernoulli parameters:

$$p_{\text{base}}, \qquad p_{\text{ft}}. \tag{11}$$

We place a uniform Beta prior on each parameter:

$$p_{\text{base}} \sim \text{Beta}(1,1), \qquad p_{\text{ft}} \sim \text{Beta}(1,1). \tag{12}$$

After observing s successes and f failures, the posterior distribution becomes

$$p \mid \text{data} \sim \text{Beta}(1+s, 1+f). \tag{13}$$

Thus, for the baseline and fine-tuned models, we obtain

$$p_{\text{base}} \mid \text{data} \sim \text{Beta}(1+s_{\text{base}}, 1+f_{\text{base}}), \tag{14}$$

$$p_{\text{ft}} \mid \text{data} \sim \text{Beta}(1+s_{\text{ft}}, 1+f_{\text{ft}}). \tag{15}$$

The main quantity of interest is the posterior probability that the fine-tuned model has a higher true success rate than the baseline model:

$$P(p_{\text{ft}} > p_{\text{base}} \mid \text{data}). \tag{16}$$

We also compute the posterior distribution of the improvement

$$\Delta p = p_{\text{ft}} - p_{\text{base}}, \tag{17}$$

and report its 95% credible interval. This allows us to distinguish small observed improvements from improvements that are more strongly supported by the data.

9. Results

The results are reported on three datasets: the main test set, the Spanish probe set, and the Portuguese probe set. The probe sets are used to evaluate whether fine-tuning transfers to languages that were not the primary focus of training.

Overall, the fine-tuned model improves over the baseline in most settings. However, the evidence is strongest for embedding-based semantic similarity, while exact-match improvements remain small.

9.0.1. Main Test Set

Table 2 reports the Bayesian comparison on the main test set.

Metric	Base	FT	Δ	$P_{\mathrm{FT>Base}}$	95% CI
Exact match	9/612	13/612	0.007	0.800	$[-0.009, 0.022]$
Token match	9/612	15/612	0.010	0.888	$[-0.006, 0.026]$
Embedding match	241/612	289/612	0.078	0.997	$[0.023, 0.133]$

Table 2. Bayesian comparison between baseline and fine-tuned model on the main test set.

On the main test set, the fine-tuned model improves across all three metrics. The exact-match gain is small, increasing from 9 to 13 successes out of 612 examples. The posterior probability that the fine-tuned model is better is 0.800, but the credible interval still overlaps zero.

The token-match result is slightly stronger, with 15 successes for the fine-tuned model against 9 for the baseline. However, the credible interval again includes zero.

The clearest improvement is obtained with embedding match. The fine-tuned model increases from 241 to 289 successes, with a posterior probability of 0.997 and a fully positive credible interval. This suggests that fine-tuning substantially improves semantic proximity to the target answers on the main test set.

This result should still be interpreted carefully. Embedding similarity measures semantic closeness, not factual correctness. It can reward answers that are close in topic but wrong in important details. Therefore, the embedding result shows that the model moves closer to the target distribution, not that all its answers are correct.

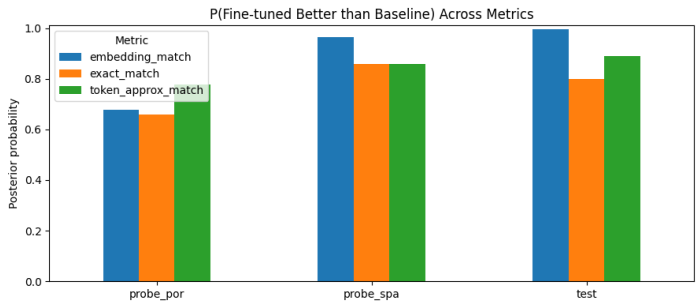

Figure 5. Posterior comparison of semantic-similarity success rates for the baseline and fine-tuned models. The distributions represent uncertainty over the true success probability after observing the embedding-match outcomes. Separation between the two posterior distributions indicates stronger evidence that fine-tuning improves semantic proximity to the target answers.

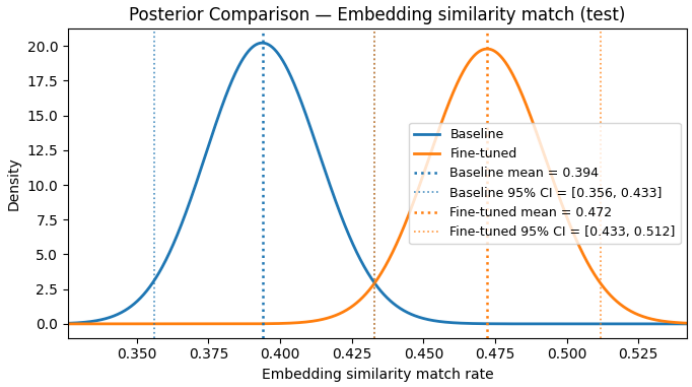

Figure 6. Posterior probability that the fine-tuned model has a higher true success rate than the baseline model, reported across datasets and metrics. Values closer to one indicate stronger Bayesian evidence in favour of the fine-tuned model. This plot summarizes how the strength of the fine-tuning effect depends on both the dataset and the evaluation criterion.

9.1. Spanish Probe Set

Table 3 reports the results on the Spanish probe set.

Metric	Base	FT	Δ	$P_{\mathrm{FT>Base}}$	95% CI
Exact match	2/300	5/300	0.010	0.858	$[-0.009, 0.030]$
Token match	2/300	5/300	0.010	0.858	$[-0.009, 0.030]$
Embedding match	75/300	95/300	0.067	0.965	$[-0.006, 0.138]$

Table 3. Bayesian comparison between baseline and fine-tuned model on the Spanish probe set.

The Spanish probe set shows a similar but weaker pattern. Exact match and token match both improve from 2 to 5 successes out of 300 examples. The posterior probability of improvement is 0.858, but the credible interval still overlaps zero.

The embedding metric gives stronger evidence. The fine-tuned model improves from 75 to 95 successes, with a posterior probability of 0.965. The credible interval is mostly positive, although it still slightly overlaps zero.

This suggests a possible cross-lingual transfer effect. Since Spanish is used as a probe language, the improvement indicates that fine-tuning may generalize beyond the main training distribution. However, because the strongest signal comes from the embedding metric, this conclusion should remain cautious.

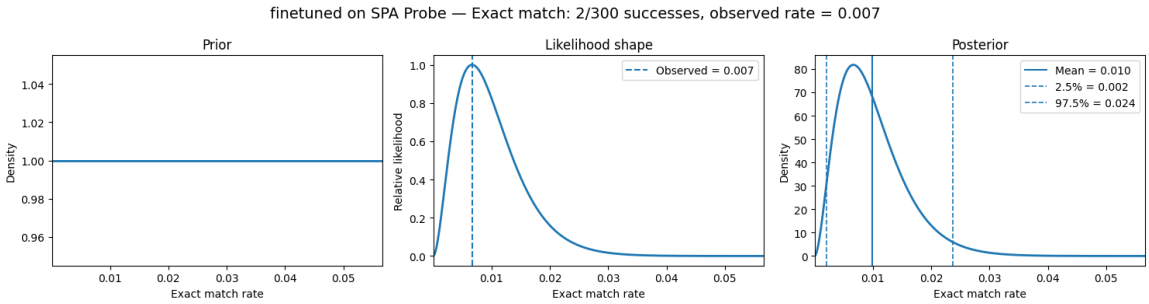

Figure 7. Bayesian prior, likelihood shape, and posterior distribution for the fine-tuned model on the Spanish probe set. A uniform Beta$(1, 1)$ prior is updated using the observed number of successes and failures. The posterior summarizes uncertainty over the true success probability of the fine-tuned model on Spanish examples.

9.2. Portuguese Probe Set

Table 4 reports the results on the Portuguese probe set.

Metric	Base	FT	Δ	$P_{\mathrm{FT>Base}}$	95% CI
Exact match	2/300	3/300	0.003	0.659	$[-0.014, 0.021]$
Token match	2/300	4/300	0.007	0.776	$[-0.011, 0.026]$
Embedding match	80/300	85/300	0.017	0.677	$[-0.054, 0.087]$

Table 4. Bayesian comparison between baseline and fine-tuned model on the Portuguese probe set.

The Portuguese probe set gives the weakest evidence of improvement. Exact match increases only from 2 to 3 successes, and token match from 2 to 4 successes. In both cases, the posterior probability of improvement remains moderate and the credible intervals clearly include zero.

Embedding match also shows only a small increase, from 80 to 85 successes. The posterior probability is 0.677, with a wide credible interval. Therefore, the Portuguese results do not support a strong claim of improvement.

This dataset is useful as a control case. It shows that the effect of fine-tuning is not uniform across all related languages.

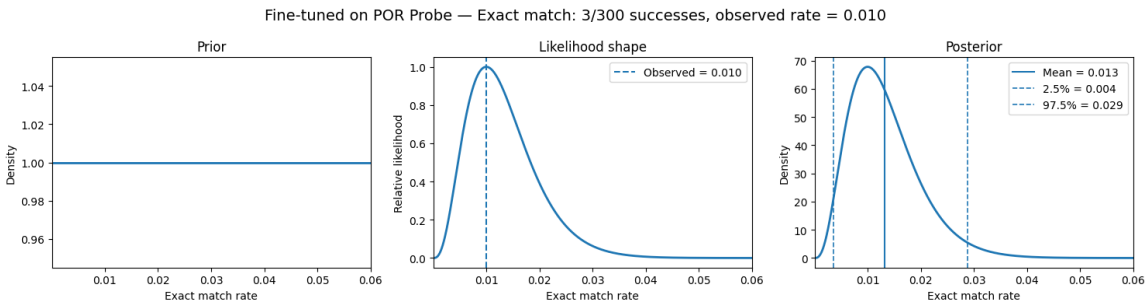

Figure 8. Bayesian prior, likelihood shape, and posterior distribution for the fine-tuned model on the Portuguese probe set. A uniform Beta$(1, 1)$ prior is updated using the observed number of successes and failures. The posterior summarizes uncertainty over the true success probability of the fine-tuned model on Portuguese examples.

9.3. Cross-Metric Interpretation

The results show a consistent but nuanced pattern. The fine-tuned model usually improves over the baseline, but the strength of the improvement depends on the metric.

The strongest evidence appears under embedding match. This indicates that fine-tuning moves the generated answers closer to the target answers in semantic space. The effect is clear on the main test set and visible on the Spanish probe set.

Exact match and token match show smaller gains. This is expected for generative outputs, since valid answers may differ from the reference in wording or formatting. These stricter metrics suggest that fine-tuning does not reliably make the model reproduce the exact expected answer.

The gap between lexical and semantic metrics is important. It suggests that fine-tuning mainly improves answer relevance and semantic alignment, rather than exact target reproduction. In this sense, the fine-tuned model behaves as if it has shifted toward the target multilingual answer distribution, while still preserving its own generative style.

Finally, the Portuguese probe set prevents an overly broad conclusion. The weak results

there show that the multilingual benefit is not uniform across all related languages.

10. Discussion and Conclusion

The evaluation suggests that supervised fine-tuning with LoRA produces a measurable behavioural shift in the model. The improvement is not dramatic under strict exact-match evaluation, but it becomes more visible when using softer lexical and semantic metrics.

Main result. The clearest improvement appears on the main test set. Under the embedding-match metric, the fine-tuned model improves from 241 to 289 successes out of 612 examples. The posterior probability that the fine-tuned model has a higher true success rate than the baseline is 0.997, and the 95% credible interval for the improvement is entirely positive. This provides strong evidence that fine-tuning makes the model outputs semantically closer to the reference answers.

By contrast, the exact-match improvement is much smaller. On the same test set, exact match increases only from 9 to 13 successes. The posterior probability of improvement is 0.800, but the credible interval still overlaps zero. Therefore, the strict correctness claim should remain limited: the fine-tuned model does not reliably learn to reproduce the exact target answers.

Probe-language behaviour. The probe-language results are informative for evaluating cross-lingual transfer. On Spanish, the model shows some evidence of improvement, especially under embedding similarity. This suggests partial generalization beyond the main training distribution, although the evidence is weaker than on the main test set.

Portuguese acts as a useful control case. Improvements are small across all metrics, and the Bayesian posterior does not support a strong claim that the fine-tuned model is clearly better. This prevents an overly broad conclusion that fine-tuning uniformly improves multilingual performance across related languages.

Length and answer style. The length analysis adds an important behavioural interpretation. The fine-tuned model produces shorter answers and reduces mean absolute length error across all datasets. This suggests that fine-tuning affects not only answer content, but also response style and format.

In particular, the model appears to move toward a more concise answer pattern. This likely contributes to the observed improvements in token overlap and semantic similarity, since shorter and more direct answers are closer to the expected target format.

Interpretation. The main conclusion is therefore nuanced. Fine-tuning improves the model's semantic proximity to the target answers and makes its outputs more aligned with the expected response format. The strongest evidence appears on the main test set, with moderate evidence of transfer to Spanish and weak evidence for Portuguese.

At the same time, automatic metrics alone are not sufficient to establish factual correctness. Exact match is often too strict, while embedding similarity can be too permissive: it may reward answers that are topically close but wrong in key details. For this reason, the automatic evaluation should be interpreted as a diagnostic layer rather than as a final correctness judgment.

The most accurate interpretation is that fine-tuning acts as a controlled attribute shift. It changes the model's behaviour toward the target multilingual answer distribution, making responses shorter, more relevant, and more semantically aligned. However, it does not provide evidence of a uniform increase in general reasoning or factual ability.

Future work. Future work should include a larger manual evaluation set, especially for borderline cases where embedding similarity is high but exact match fails. A more robust evaluation could also separate different error types, such as factual errors, language errors, verbosity errors, and formatting errors. This would make it possible to determine whether the observed semantic improvement corresponds to genuinely better answers or mainly to better stylistic alignment.